

第4回: OpenAI APIに初めて触れる

LLMアプリケーション基礎

今日のゴール

Python から OpenAI API を呼んで、返答が返ってくる体験をする

今日の流れ

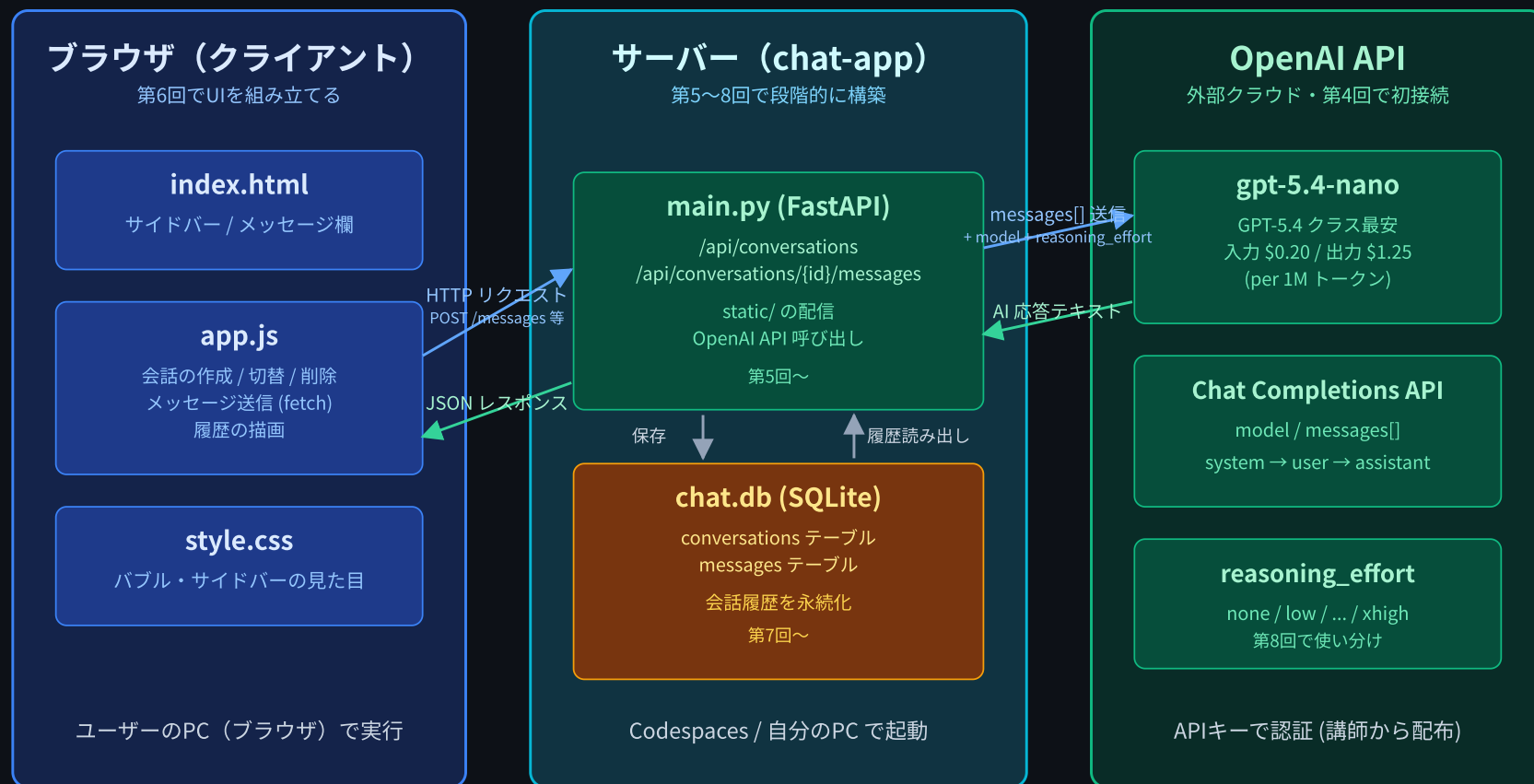
前半

- OpenAI API とは / 料金の見方
- 使うモデル: `gpt-5.4-nano`
- APIキーの配布と **安全な管理**
- `pip install openai`

後半

- 最小スクリプトで1往復してみる
- `model` / `messages` の意味
- **Reasoning**（考えてから答える）を体感する — `none` vs `high`
- トークンとコストの見方
- 演習: CLIチャットを書く

chat-app の全体像

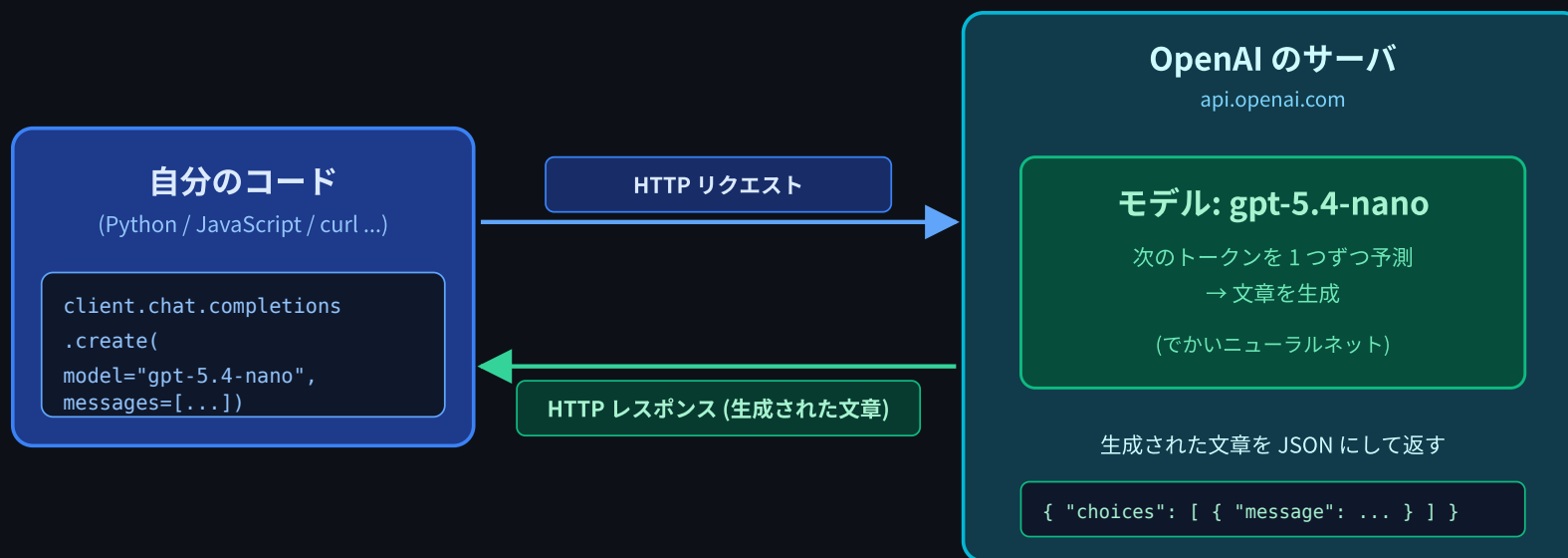


今日は右の **OpenAI API** に初接続 する (サーバーもブラウザもまだ無し、Pythonから直接)

OpenAI API とは

- ChatGPT の中で動いている **モデルそのもの** に、HTTP越しに話しかけるための仕組み
- 自分のプログラムから呼べる = 自分のアプリに AI を組み込める
- 言語は何でもよい(Python / JavaScript / curl ...)。今日は Python SDK を使う

OpenAI API: HTTP 越しに 1往復



サーバーもブラウザもまだ無し — Python から直接 OpenAI に話しかける

料金体系の概要

OpenAI API は 使った分だけ 課金される(従量制)

- 単位は トークン(文章を細切れにした単位。1トークン ≡ 日本語1～2文字)
- 入力トークン(送った分)と 出力トークン(返ってきた分)で別々に値段が付く
- モデルが高性能ほど高い

今日のメインモデル `gpt-5.4-nano` は、GPT-5.4 クラスで 最も安い ライン

今日使うモデル: `gpt-5.4-nano`

項目	値
入力料金	\$0.20 / 1Mトークン
出力料金	\$1.25 / 1Mトークン
コンテキストウィンドウ	400K トークン
最大出力	128K トークン
Reasoning	対応(<code>none</code> ~ <code>xhigh</code>)

- 「安い + 大きな窓 + 推論対応」のバランス型
- 普段のチャット用途には十分高品質
- 「うっかり爆発」しても 講師側で月予算上限 を設定済み

APIキー の配布

- 講師から配布する(受講生個人での取得は不要)
- 配布方法は当日アナウンス
- 受け取ったキーは:
 - 他人に見せない
 - チャット/メールで送らない
 - スクリーンショットに写さない
 - **GitHub** にコミットしない(これが最大の事故元)

上限は設定してあるけど、無駄な消費はしない意識を持つ

APIキーの安全な管理 ①: シェルの環境変数

毎回 起動前に、シェルで `export` する

```
export OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxx  
python chat.py
```

- `OPENAI_API_KEY` は OpenAI SDK が 自動で読みに行く 環境変数名
- コードに `sk-...` を書かなくて済む
- `git add` 対象に 絶対にならない

export のスコープに注意

```
$ export OPENAI_API_KEY=sk-xxx
$ python chat.py          # ← OK、使える

$ # ターミナルを閉じる / 新しいタブを開く

$ python chat.py          # ← もうダメ。AuthenticationError
```

- `export` は 今開いているシェルにだけ 有効
- 新しいタブ・新しいセッションでは やり直し
- 「面倒くさい」けどこれが 安全側に倒した仕様
 - キーがファイルとして残らない = 流出リスクが減る

なぜ `.env` ファイルを使わないのか？

世の中には `.env` というファイルにキーを書く方式もある

```
# .env
OPENAI_API_KEY=sk-xxx
```

- これ自体は便利
- ただし `.gitignore` 設定を1行忘れた瞬間に **GitHubにキーが流出する事故** が頻発
- 本コースでは **毎回手で `export` に統一する**
 - キーがファイルに残らない = 事故が起きない

やってはいけないこと

絶対NG

1. コードに直書き: `client = OpenAI(api_key="sk-xxxxx")`
2. GitHub に push する(public/private 問わず)
3. ブラウザ側の JavaScript に書く (F12で誰でも見える)
4. Slack や DM、メールで生のキーを共有する
5. スクリーンショットに写ったまま投稿する

GitHub は流出キーを自動検出して通知してくれるが、その前に第三者が使い切る事例は多い

ライブラリのインストール

```
pip install openai
```

devcontainer で起動した環境なら、すでに入っているはず

```
$ python -c "import openai; print(openai.__version__)"  
1.xx.x
```

確認できれば準備完了

最小スクリプト: 1往復してみる

```
# one_shot.py
from openai import OpenAI

client = OpenAI() # 環境変数 OPENAI_API_KEY を自動で読む

response = client.chat.completions.create(
    model="gpt-5.4-nano",
    messages=[
        {"role": "user", "content": "こんにちは。自己紹介してください。"},
    ],
)

print(response.choices[0].message.content)
```

```
$ export OPENAI_API_KEY=sk-xxx
$ python one_shot.py
こんにちは。私はAIアシスタントの...
```

messages 配列の中身

```
messages = [  
    {"role": "system", "content": "あなたは親切で丁寧なアシスタントです。日本語で回答してください。"},  
    {"role": "user", "content": "Pythonとは?"},  
    {"role": "assistant", "content": "Pythonとは..."},  
    {"role": "user", "content": "じゃあJavaScriptは?"},  
]
```

- **system** : AI の役割・性格を指示(冒頭1回)
- **user** : 人間の発言
- **assistant** : AI の過去の発言(履歴を渡すときに使う)

第6回でこの配列に履歴を積んでマルチターン会話を作る

レスポンスから何が取れる？

```
response = client.chat.completions.create(...)

# 本文
print(response.choices[0].message.content)

# トークン使用量
print(response.usage.prompt_tokens)      # 送った分
print(response.usage.completion_tokens)  # 返ってきた分
print(response.usage.total_tokens)       # 合計
```

usage を確認する習慣 を最初から付けておくとコスト感が掴める

「考えてから答える」モデル: Reasoning

第1回で名前だけ紹介した話の 本編

- `gpt-5.4-nano` は 推論対応モデル
- API呼び出し時に どれくらい考えるか を選べる
- パラメータ名: `reasoning_effort`

値	動き
<code>none</code>	推論しない。最速・最安
<code>low</code>	少しだけ考える
<code>medium</code>	そこそこ考える
<code>high</code>	しっかり考える
<code>xhigh</code>	限界まで考える

Reasoning の使い方

```
response = client.chat.completions.create(  
    model="gpt-5.4-nano",  
    messages=[  
        {"role": "user", "content": "難しい論理パズル..."},  
    ],  
    reasoning_effort="high",    # ← ここ  
)
```

たった **1行追加** するだけで、モデルの挙動が変わる

デモ: `none` vs `high` で同じ質問

同じ難問を両方で投げしてみる(`reasoning_demo.py`)

例題:

太郎は花子の弟である。次郎は太郎の父である。三郎は次郎の兄である。
花子から見て三郎は誰か？

- `none` ... 速い / 安い / でも難しい問題は外しがち
- `high` ... 遅い / 高い / 正答率は上がる

比較結果(イメージ)

reasoning_effort: 「どれくらい考えるか」を1行で切替



reasoning_effort = "none"

```
client.chat.completions.create(
```

```
  reasoning_effort="none", ... )
```

レイテンシ: 0.8 秒

思考トークン: 0

合計トークン: ≈100 (安い)

向き: 雑談 / 要約 / 翻訳 / 分類

難問: 外しがち

reasoning_effort = "high"

```
client.chat.completions.create(
```

```
  reasoning_effort="high", ... )
```

レイテンシ: 6.2 秒

思考トークン: 数百～数千 (課金対象)

合計トークン: ≈数千 (高い)

向き: 論理パズル / 数学 / コード難読バグ

難問: 正解しやすい

推論トークンも課金対象。 `high` を使う場面は選ぼう

使い分けの指針

`none` / `low` を使う

- 雑談
- 短い要約
- 単純な質問応答
- 速度重視のチャット
- 大量バッチ処理

`high` / `xhigh` を使う

- 論理パズル / 数学
- コードレビュー・難読バグ
- 複雑な計画立案
- 高品質な作文の最終仕上げ

デフォルトは安い側。難しい時だけ高くする、が基本戦略

トークン消費とコストの計算

`gpt-5.4-nano` の料金:

- 入力: \$0.20 / 1Mトークン
- 出力: \$1.25 / 1Mトークン

例: 1回の往復で `prompt_tokens=200` , `completion_tokens=300` だった場合

```
入力: 200 / 1,000,000 * $0.20 = $0.00004  
出力: 300 / 1,000,000 * $1.25 = $0.000375  
合計: 約 $0.0004 = およそ 0.06円
```

1回ならほぼ無料。ただし 会話が長くなると毎回過去全部を送り直す ことを忘れずに(第6回で扱う)

演習: CLIチャット (`chat.py`)

ターミナルで AI と対話できるスクリプトを作る

```
$ export OPENAI_API_KEY=sk-xxx
$ python chat.py
あなた: こんにちは
AI: こんにちは!今日はどうしましたか?
あなた: Pythonの内包表記を教えて
AI: 内包表記は ...
あなた: さっき教えてくれた書き方で偶数だけ取り出して
AI: はい、先ほどの内包表記を使うと ... ← 履歴を覚えている
あなた: exit
```

要件:

- `input()` でユーザーの入力を受ける
- `messages` 配列に履歴を積んで送る(マルチターン)
- `exit` か `Ctrl+C` で終了

ヒント: `chat.py` の骨格

```
from openai import OpenAI

client = OpenAI()
messages = [
    {"role": "system", "content": "あなたは親切で丁寧なアシスタントです。日本語で回答してください。"},
]

while True:
    user_input = input("あなた: ")
    if user_input.strip() == "exit":
        break
    messages.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-5.4-nano",
        messages=messages,
    )
    reply = response.choices[0].message.content

    print(f"AI: {reply}")
    messages.append({"role": "assistant", "content": reply})
```


本日のまとめ

学んだこと

1. **OpenAI API** は HTTP越しにモデルに話しかける仕組み
2. 使った トークン分だけ課金 される (`gpt-5.4-nano` は安いライン)
3. **APIキー**はシェルの環境変数 で渡す。コードに書かない・コミットしない
4. `chat.completions.create(model, messages, ...)` で1往復
5. `messages` 配列に履歴を積めば **マルチターン**
6. `reasoning_effort` で「考える深さ」を選ぶ (`none` ～ `xhigh`)
7. デフォルトは安い側、難しい時だけ高くする

次回予告

第5回: FastAPIでChatバックエンドを作る

今日作った CLI を **HTTP API** に変える。 `POST /api/chat` を FastAPI で実装し、なぜブラウザから直接 OpenAI を呼ばないのか（APIキー保護）も扱う。いよいよ `chat-app/` 本体の構築開始。

提出物

実習で作成したファイルをフォームから提出してください:

1. `chat.py` の GitHub のURL

◦ 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session04/chat.py`

お疲れ様でした！